

Stress management project report

Overview:

A Scala console chatbot that helps users manage stress through a menu-driven interface. It detects stress types from user messages, suggests breathing techniques or relaxing activities, and tracks the conversation to provide mood detection, topic summaries, and interaction history.

First: Core chatbot

Classes and others:

“case class Type”: this class contains information about the stress management technique, and it mainly contains the name of the technique, the stress type that matches the technique and the list that contains the steps of the technique.

“case class StressChatbot”: this is the main chatbot class. It contains a list that saves the messages of the user and a list containing both the stress type and the technique for each stress type.

“Case class Activity”: upon the user’s mood and the engine chooses his activity; this class is responsible for getting the name of the activity, the type or the category that this activity is included in, the duration of this activity and the mood it aims for.

“Object RecommendationEngine”: It’s a singleton object that’s responsible for all recommendation logic, it has the list of activities that covers different names, categories, durations and moods.

“userPreference”: a mutable map that stores user’s preferences as key value pairs.

Functions:

“greetUser()”: this function is responsible for welcoming the user and offering the user ways to help him

“handleUserInput(input)”: this function is responsible for handling user input as it calls **parseInput(input) function** and

generateResponses(words) function and saves the input in the history and also returns a response for the user.

“parseInput(input)”: this function splits the user’s raw message and save them into tokens and save them in a cleaned-lowercased-wordlist

“generateResponse(words)”: this function is responsible for replying on the user by matching the cleaned keywords based on the user’s messages.

“formatType(t)”: a helper function that generates numbered list “as bullet points” to number the steps in the technique.

“getUserPreference()”: it reads the stores preferences in **“userPreference”** map and returns them.

“updatePreference(key, value)”: it takes the user’s preferences and stores them in map form, and it saves the updated map in **“userPreference”**.

“recommend(preferences, data)”: it goes over the mood in the preferences map and filters the activity list in order to find the activity that matches the user’s mood and it sorts the activities ascendingly based on their durations.

“explainRecommendation()”: this function describes why the chatbot chooses this activity and why finds it suitable for the user’s case.

Second: smart conversation memory and context tracker:

Classes and others:

“Case class InteractionEntry”: this class is responsible for recording each single exchange between the chatbot and the user. It stores the number of message’s turn, the date and time of the message, the user’s input, the chatbot response and the detected mood of the user

“case class ConversationState”: this class represents the whole conversation’s state. It stores all the conversation history, user’s preferences, the user’s current tone or mood

Functions:

“logInteraction(userInput, botResponse, context, detectedIntent)”:

This function is responsible for recording and saving each new exchange between the user and the chatbot, it saves it as a new interaction entry to the history by incrementing the history size (sequence number) automatically and updates the current time stamp ,and finally it returns a new updated copy of the new interaction entry.

“getConversationHistory(context)”: this is a getter function that is responsible for returning the all the history interactions and previous conversations between the chatbot and the user that are stores in conversation state.

“getLastNInteractions(n, context)”: this function returns only the last N interactions by using **takeRight(n) function**, which is useful for showing a recent conversation summary.

“detectRepeatedQuery (input, history)”: this function is responsible for normalizing the user’s input (lowercase-trimmed-input) bot the previous and the new user’s inputs, then it checks if both are similar, it returns True else returns false.

“detectIntent(input)”: this function is responsible for splitting the user’s message into a named intent to find the appropriate keyword that matches the user’s intent and then returns the type of intent that the user had.

“extractTopics(history)”: this function is responsible for iterating over each interaction in the history and extracts the important and meaningful words from the user’s input.it iterates over each entry until Nil and then each entry it iterates over it, it cleans its text and passes the words in

the helper function **“Filterwords”** that keeps only words more than 3 characters.

“getMostDiscussedTopics(history)”: this function is responsible for finding the top 5 most frequent intents discussed and return a list od intent name and count pairs.

“summarizeConversation(history)”: this function is responsible for returning a readable summary for the user’s conversation.

“getLastMessage(history)”: This function is responsible for going to the end of conversation history and returning the last message. it returns the message in lowercase.

“getUserMood(history)”: this function is responsible for detecting user’s mood from the last message by checking if it contains any word from two predefined lists. And it whether returns **negative** or **neutral**.

Third: the “main” function:

It ties every function together, the main loop that ties everything together. It creates a Stress Chatbot, initializes a Conversation State, and runs a loop showing a menu.

Fourth: Bonus:

“case class GameQuestion”: This class is responsible for storing the stressful situation presented for the user, a list of choices, the number of correct choices and the explanation for why this is the correct answer.

“getGameQuestions()”: this function is responsible for returning a phrased list of stress scenario questions covering different situations.

“askGameQuestion(question, questionNumber)”: this function is responsible for taking each question at a time, as it prints the situation and numbered choices and reads the user’s answer. If correct it returns 1 and the explanation and prints correct, if wrong it returns 0 and the explanation and prints the correct answer.

“showGameResult(score, total)”: this function is responsible for displaying the final score and a message based on the user’s performance.

“playStressReliefGame()”: this is the main game function which connect every function together.